Assignment 8 – Minimum Spanning Trees using Prim's Algorithm

Submit zip file to Canvas by 11:59pm November 11.

Goal:

You can learn a lot about algorithms by implementing them. Implementing an algorithm clarifies the problem being solved and can make complexity analysis an easier task.

Assignment:

You must create an implementation of Prim's algorithm for finding a minimum spanning tree in a weighted, undirected graph using Python 3. Prim's algorithm is detailed in Cormen 23.2 on page 634.

Specifically, you must complete the functions `compute_mst` and `write_tree_edges_to_file` in the `prim_mst.py` file. The skeleton of these functions is included in the supplied file. You must also provide time tests of your implementation, along with a statement about why or why not your implementation met or did not roughly meet the expected time complexity with different input sizes as discussed below.

See the terminal screen example in the section below for the commands we will use to test your program.

Canvas Submission:

You must submit the following items in a zip file:

- prim_mst.py
- any additional files it imports
- a .pdf file that shows the results of timing your program on different sized inputs and a brief discussion of how well you think your implementation meets the theoretical complexity bounds for the algorithm based on input size (see *Time Complexity* section below for details).

BE SURE TO UNZIP YOUR SUBMISSION ON THE SCHOOL MACHINES AND RUN YOUR PROGRAM. If your program fails to run on the school machines with our tests, you will receive a zero. Your complexity analysis file must be a .pdf file. We will not be grading any other type of file. You must also use the class GitHub site.

When you are finished, you'll have a zip file that can be used like this:

```
17:16 cole@csf-spectre % ll
total 8.0K
-rw-r--r-- 1 cole users  134 Oct 20 15:17 some_graph_to_test.txt
-rw-r--r-- 1 cole users 1.8K Oct 20 17:12 u0xee-prim-mst.zip
17:16 cole@csf-spectre % unzip u0xee-prim-mst.zip
Archive:  u0xee-prim-mst.zip
  inflating: prim_mst.py
  inflating: undirected_graph.py
17:16 cole@csf-spectre % ll
total 16K
-rw-r--r-- 1 cole users 1.3K Oct 20 15:05 prim_mst.py
-rw-r--r-- 1 cole users  134 Oct 20 15:17 some_graph_to_test.txt
-rw-r--r-- 1 cole users 1.8K Oct 20 17:12 u0xee-prim-mst.zip
-rw-r--r-- 1 cole users 2.9K Oct 20 16:54 undirected_graph.py
17:16 cole@csf-spectre % python3 -i prim_mst.py
>>> compute_mst("some_graph_to_test.txt")
>>>
17:17 cole@csf-spectre % cat some_graph_to_test.txt.mst
v0 v4 3
v4 v7 0
v4 v5 1
v4 v1 2
v0 v3 4
v4 v2 5
v3 v6 6
17:17 cole@csf-spectre %
```

Note that the zip file in the above example only contained Python source files. This was because I (Cole) wanted to show you the smallest example of it working. Your zip will have more files in it.

Partial Rubric:

- We must be able to use your zip file in the manner shown above to compute a minimum spanning tree for an input graph file.
- You may not use any of the code in the reference implementations in your submission.
- We will only run your code on graphs that have a unique minimum spanning tree. Make sure your code does this correctly.
- We expect your program to perform at near-expected complexity bounds, so we will test your program on both small and huge problem sizes.
- Write your code expecting that it is important enough for other people to read. We will read your code and reward your efforts to make it good.
- We expect you to use appropriate and efficient data structures and design in your implementation.
- We expect you to use the class GitHub site as a remote repository and push your code to it periodically.
- We reserve the right to change this rubric as necessary.

Download Helper Files:

Processing graphs is not a trivial task, so we've given you starting code to take care of the less interesting bits so that you can focus on the mechanics of the algorithm itself. The starting code is available on the course website. Read it over and understand the utility of each function.

Go to the Assignments tab on the CS320 webpage, and use the link described for this assignment to download a zip file containing the skeleton code for Prim's algorithm and several helper files:

- `prim_mst.py` – skeleton code for Prim's algorithm – DO NOT change function arguments
- `undirected_graph.py` – graph class and its functions – you can use it but this is not required
- `generate_mst_input.py` – generates input graphs for you to use in testing/timing
- `undirected_graph_files_equal.py` – as the name implies to be used for testing
- `kruskal_mst_reference_implementation.py` – Kruskal's MST algorithm that you can use to generate a known MST you can test your code against.
- `sql_mst_reference_implementation.py` – an SQL implementation you can use to generate an MST to test your code against

The Kruskal's and SQL provided code compute the minimum spanning tree of a graph. This is exactly the same functionality as the code you are writing, what luck! Use this provided implementation as a **reference** to check if your code computes the correct answer for a given test case. In this way you can feel more confident in the quality of the code you have written.

You **must** implement your own version of Prim's algorithm for minimum spanning tree. It is not acceptable to reuse the reference implementation as your own. This is similar to the reason why it is not right to reuse another student's implementation as your own. Remember, we will be reading your code to see how you solved the problem.

Testing:

- Generate an input graph using the provided code
- Use your code to compute the MST, and use the provided reference implementation code to compute the MST
- Make sure these computed MSTs are the same tree, using the provided code
- Repeat these steps for various sized graphs

You should try to use your zip in the manner shown on the school machines, to make sure you've got it right.

Time complexity:

You must include a pdf file in your submission that has a plot for the time complexity of your function. Plot the running time for graphs of size 5, 50, 100, 200, 500, 1000, 2500, 5000, and

50,000. Include an explanation of the asymptotic analysis of the function, and state your reason based on the algorithm that you have implemented. Time your code using the same method described in the previous assignments.

The generated 50,000 node graph with a seed of 0 takes our reference version of Prim's under 3 seconds on school machines.

Writing good code:

You should write good code. Good code is easy for humans to understand and manipulate. This gets into the area of code style, which is important for professional programmers. See what experts think about style, such as the Google Python style guide (URL: https://google.github.io/styleguide/pyguide.html), then form your own opinions. While we'll never have perfect agreement on matters of style, it is easy to see when code is unnecessarily hard to understand. Cryptic names, dead code strewn about, nested loops, and mega-methods all make code hard to understand, change, and even write in the first place.

Good luck and have fun!